

FIT3080
Applied class: Markov Decision Processes
Sample Solutions

Exercise 1: MDPs: Basic Conceptual Questions

In the lecture, we discussed the Bellman Equation and how it can be used to characterise optimal utility in MDPs. For reference, recall this equation:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- (a) What do we call γ in this equation? Why is this parameter required? What happens if γ becomes a very large or a very small value?

[SOLUTION] γ is referred to as the discount factor, ranging a value between 0 and 1, i.e., $\gamma \in (0, 1)$. The discount factor needs to be < 1 to guarantee that our algorithms will converge and to prevent against infinite rewards for infinite state sequences, e.g., if our game lasts forever. Also, intuitively, it makes sense to value immediate rewards more highly than rewards obtained at a later time.

A smaller value of γ indicates smaller *horizon* or a shorter term focus for the consideration. If γ is close to 0, our decision only uses the very immediate rewards, e.g., the current and perhaps the next reward. If γ is closer to 1, the later rewards are almost as valuable as the earlier rewards for our planning/decision.

- (b) When does policy iteration end? Immediately after policy iteration ends (without performing additional computation), do we have the values of the optimal policy?

[SOLUTION] Policy iteration ends when the policy converges, i.e., when $\pi_{new} = \pi_{old}$ after running policy improvement.

We do have the values for the optimal policy. Since we necessarily ran policy evaluation on the most recent iteration of policy iteration, we have the value function $V^{\pi_{old}}$ corresponding to π_{old} . We know that $\pi_{new} = \pi_{old}$, implying that $V^{\pi_{new}} = V^{\pi_{old}}$. Therefore, we have $V^{\pi_{new}}$, which is the value function corresponding to the optimal policy π_{new} .

- (c) What are the major differences between the value iteration and policy iteration algorithms, and when might you prefer one to the other?

[SOLUTION] Policy iteration focuses on evaluating the policies themselves, while value iteration evaluates states or state-action pairs and implicitly derives a policy from there.

Value iteration: each iteration takes $O(|S|^2|A|)$ time (for each state s , we go over each next state s' and each action), which is a bit costly when there is a large number of states. Sometimes a policy (extract the policy as if the current V_k is V^*) has already converged to optimal, but the values have not converged, and therefore we have to continue the value iteration process, which is a waste of time.

Policy iteration: the policy evaluation step takes $O(|S|^3)$ time in the worst case (complexity of solving a system of linear equations); and the complexity of the

policy improvement step is $O(|S|^2|A|)$. Typically $|S| > |A|$, so the policy evaluation step dominates. However, policy iteration generally has fewer iterations than value iteration.

In summary, policy iteration is better than value iteration only if

$$|S^3| \times N_{PI} < |S^2| \times |A| \times N_{VI}$$

where N_{PI} and N_{VI} are the number of iterations for policy iteration and value iteration respectively.

- (d) When doing policy iteration, what changes if you only run one iteration of Bellman update during the policy evaluation step, instead of running the Bellman update until convergence? Do you still get an optimal policy?

[SOLUTION] Yes, you will still get an optimal policy. Recall that the equation for value iteration is:

$$V_{i+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i(s')]$$

And the equation for the policy evaluation step of policy iteration is:

$$V_{i+1}^{\pi_k}(s) = \sum_{s'} T(s, \pi_k(s), s') [R(s, \pi_k(s), s') + \gamma V_i^{\pi_k}(s')]$$

given some fixed policy π that we update in the policy improvement step. This policy is given by:

$$\pi_{k+1}(s) \leftarrow \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i^{\pi_k}(s')]$$

The key observation here is that this procedure is very similar to value iteration — the main difference is that value iteration uses max over actions, while this version of policy iteration (called truncated or modified policy iteration) uses the actions in the current fixed policy. In this case, we update values based on our current best policy, and keep doing that until convergence of a policy.

Exercise 2: Modified Vacuum Cleaner World

We modify the vacuum cleaner world example such that after any action, one of the two blocks will become randomly dirty with probabilities p_L and p_R for the left and right blocks respectively (Note that $p_L + p_R = 1$). Using two actions, 1) staying in the current block and vacuum (abbreviated by “S&V”) or 2) move to another block and vacuum (abbreviated by “M&V”), the vacuum cleaner should clean the dirt and receive a reward. The reward for staying in the same block and vacuum, “S&V”, (if dirty) is the inverse of the probabilities; and the reward for moving to other block and vacuum, “M&V”, (if dirty) is equal to the ratio between the target and source block probabilities. You will receive zero reward and the game ends if the block where the vacuum cleaner moves is not dirty. The goal is to maximise the cumulative sum of rewards.

- (a) Assuming you know the probabilities p_L and p_R , formulate this modified vacuum cleaner world as an MDP, writing down the reward function $R(s, a, s')$ and the transition function $T(s, a, s')$ as tables.

[SOLUTION]

States, $S = \{block_{Left}(b_L), block_{Right}(b_R), done\}$

Actions, $A = \{S\&V, M\&V\}$

Rewards:

$R(s, a, s')$ (s, a)	s'	
	b_L	b_R
$(b_L, S\&V)$	$1/p_L$	0
$(b_L, M\&V)$	0	p_R/p_L
$(b_R, S\&V)$	0	$1/p_R$
$(b_R, M\&V)$	p_L/p_R	0

Transitions:

$T(s, a, s')$ (s, a)	s'		
	b_L	b_R	done
$(b_L, S\&V)$	p_L	0	p_R
$(b_L, M\&V)$	0	p_R	p_L
$(b_R, S\&V)$	0	p_R	p_L
$(b_R, M\&V)$	p_L	0	p_R

- (b) Assume the vacuum cleaner always starts from the left block b_L and the discount factor for the Bellman equation is equal to $\gamma = p_R$. Moreover, if at the beginning, the vacuum cleaner takes the “M&V” action and receives a non-zero reward, the reward would be equal to 2. Calculate the exact values for p_L , p_R , γ , Transitions and Rewards.

[SOLUTION] According to the assumption, the vacuum cleaner starts from the left block, b_L , takes the “M&V” action and receives a non-zero reward equal to 2, i.e.,

$$R(b_L, M\&V, b_R) = \frac{p_R}{p_L} = 2 \Rightarrow p_R = 2p_L$$

Since the probabilities sum to one, i.e., $p_R + p_L = 1$,

$$2p_L + p_L = 1 \Rightarrow p_L = \frac{1}{3} \approx 0.33 \quad \text{and then} \quad p_R = \frac{2}{3} \approx 0.67$$

Similarly $\gamma = p_R \approx 0.67$ and

Rewards:

$R(s, a, s')$ (s, a)	s'	
	b_L	b_R
$(b_L, S\&V)$	3	0
$(b_L, M\&V)$	0	2
$(b_R, S\&V)$	0	1.5
$(b_R, M\&V)$	0.5	0

Transitions:

$T(s, a, s')$ (s, a)	s'		
	b_L	b_R	done
$(b_L, S\&V)$	0.33	0	0.67
$(b_L, M\&V)$	0	0.67	0.33
$(b_R, S\&V)$	0	0.67	0.33
$(b_R, M\&V)$	0.33	0	0.67

- (c) Considering $V_0(s) = 0$ and starting from b_L , calculate the value iterations over two iterations.

[SOLUTION] Recall that the equation for value iteration is:

$$V_{i+1}(s) = \max_{a \in A} \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i(s')]$$

Therefore, for the first iteration, we have:

$$V_1(s = b_L) = \max_{a \in A} \sum_{s'} T(b_L, a, s') [R(b_L, a, s') + 0.67 * V_0(s')]$$

$$\begin{aligned} V_1(s = b_L) &= \max\{0.33[3 + 0] + 0.67[0 + 0], 0.67[2 + 0] + 0.33[0 + 0]\} \\ &= \max\{1 + 0, 1.33 + 0\} = 1.33 \quad (a = M\&V) \end{aligned}$$

$$V_1(s = b_R) = \max_{a \in A} \sum_{s'} T(b_R, a, s') [R(b_R, a, s') + 0.67 * V_0(s')]$$

$$\begin{aligned} V_1(s = b_R) &= \max\{0.67[1.5 + 0] + 0.33[0 + 0], 0.33[0.5 + 0] + 0.67[0 + 0]\} \\ &= \max\{1 + 0, 0.167 + 0\} = 1 \quad (a = S\&V) \end{aligned}$$

For the second iteration, we have:

$$V_2(s = b_L) = \max_{a \in A} \sum_{s'} T(b_L, a, s') [R(b_L, a, s') + 0.67 * V_1(s')]$$

$$\begin{aligned} V_2(s = b_L) &= \max\{0.33[3 + 0.67 \times 1.33] + 0.67[0 + 0], 0.67[2 + 0.67 \times 1] + 0.33[0 + 0]\} \\ &= \max\{1.284, 1.79\} = 1.79 \quad (a = M\&V) \end{aligned}$$

$$V_2(s = b_R) = \max_{a \in A} \sum_{s'} T(b_R, a, s') [R(b_R, a, s') + 0.67 * V_2(s')]$$

$$\begin{aligned} V_2(s = b_R) &= \max\{0.67[1.5 + 0.67 \times 1] + 0.33[0 + 0], 0.33[0.5 + 0.67 \times 1.33] + 0.67[0 + 0]\} \\ &= \max\{1.45, 0.46\} = 1.45 \quad (a = S\&V) \end{aligned}$$

Exercise 3: Value Iteration for the Maze (Python Code)

Recall the “Example: Value Iteration” from the week 8 lecture materials.

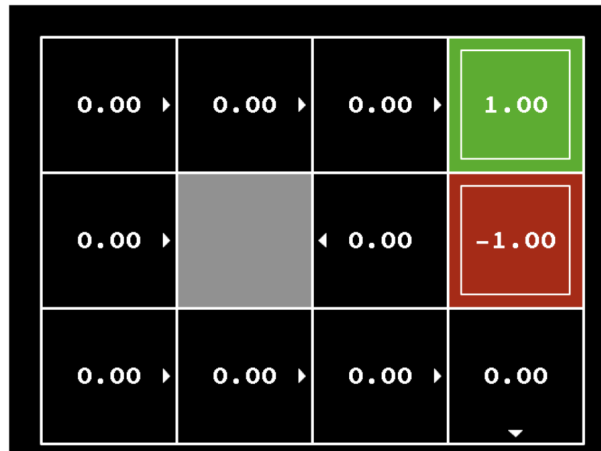


Figure 1: Maze Structure

In this exercise, you are required to implement the *Value Iteration* algorithm using the Bellman equation for the given maze (Fig. 1). The maze has two terminal states (i.e., green and red blocks), and there are no rewards on any other states. You need to write a Python code that performs the following tasks:

- Implement the Value Iteration algorithm with a noise factor of 0.2 and a discount factor (γ) of 0.9.
- Calculate the values and policy for each block of the maze.
- Compare your calculated values and policy with those discussed in the lecture.

[SOLUTION] See associated code.